

link/Index.mjs

Your main Lambda is:

```
1 | umar-coldchain-LinkDynamoDB-terraform
```

and its four layers are:

```
1 | index.mjs
2 |   ↓
3 | readingsController.mjs
4 |   ↓
5 | readingsService.mjs
6 |   ↓
7 | readingsRepositories.mjs
```

Let's go through every one.

link-dynamodb/index.mjs

This is the smallest JavaScript file.

It looks roughly like:

```
1 | import {
2 |   handleRequest
3 | } from "../controller/readingsController.mjs";
4 |
5 |
6 | export const handler = async (
7 |   event,
8 |   context
9 | ) => {
10 |
11 |   return handleRequest(
12 |     event,
13 |     context
14 |   );
15 | };
```

Block 1 — Importing the controller

```
1 | import {
2 |   handleRequest
3 | } from "../controller/readingsController.mjs";
```

This says:

I need the `handleRequest` function from my controller.

So:

```
1 | index.mjs
2 |   |
3 |   | import
```

```
4 ▼  
5 controller/readingsController.mjs
```

NOTE: An AWS Lambda handler is the specific method or function in your code that acts as the entry point to start execution when your Lambda function is invoked.

Block 2 — Lambda handler

```
1 export const handler = async (  
2   event,  
3   context  
4 ) => {
```

This is extremely important.

Terraform tells AWS:

```
1 handler = "index.handler"
```

That means:

```
1 index  
2   ↓  
3 index.mjs  
4  
5 handler  
6   ↓  
7 export const handler
```

AWS therefore starts here

What are event and context ?

AWS gives your function these automatically.

event

Contains the request.

For example:

```
1 HTTP method  
2 path  
3 branchId  
4 readingId  
5 query parameters  
6 request body
```

So if Postman sends:

```
1 PATCH  
2 /branches/BRANCH-001/readings/ABC
```

the event contains information similar to:

```
1 event.httpMethod
```

```
2
3 event.resource
4
5 event.pathParameters.branchId
6
7 event.pathParameters.readingId
8
9 event.body
```

context

Contains information about the Lambda execution.

For example:

```
1 context.awsRequestId
```

That gives each Lambda invocation a unique request ID which is useful for CloudWatch logging.

What `index.mjs` actually does

It receives:

```
1 AWS request
```

and immediately passes it here:

```
1 return handleRequest(
2   event,
3   context
4 );
```

So:

```
1 API Gateway
2   ↓
3 Lambda
4   ↓
5 index.mjs
6   ↓
7 handleRequest()
8   ↓
9 readingsController.mjs
```

`index.mjs` doesn't care whether this is POST, GET or DELETE.

That's the controller's job.